AI-Driven Real-Time Data Synchronization in Distributed Financial Systems

Karri Sairamakrishna BuchiReddy
Lead Software Developer
Systems Technology Group, Inc Farmington,
MI, 48335
sairamakrishna.karri@gmail.com

Phaneendra Siddana
Senior Software Engineer
SS Software Solutions LLC
Richmond, VA, 23294
phaneendrasiddana@gmail.com

Sandeep Kumar Srivastava
Lead Software Engineer
Capital One Financial
Richmond, VA, 23233
sandeep0887@gmail.com

Ramireddy Chilakala
Electrical and Electronics Engineering,
VNR Vignana Jyothi Institute of Engineering and Technology, Hyderabad- India
crreddy229@gmail.com

Abstract - With distributed financial microservices, the Dual-Write problem frequently results in data discrepancy between payment gateways and in-house ledgers. Conventional reconciliation schemes are based on high-latency batch reconciliation or hard-coded rules, and cannot identify Soft Drifts, small corruptions in the data (e.g. 3% deviation) that resemble normal variance. The paper suggests a real-time reconciliation model that combines an Apache Kafka streaming high-throughput system and an Unsupervised Isolation Forest anomaly detector. The experimental outcomes have shown that although the application of static rules resulted in a Recall rate of only 52.1% (it does not detect soft drifts), the offered AI model attained 100% Recall in all types of drifts. Moreover, the system had a consistent latency of 3.76 ms which was found to be viable in high-frequency trading settings where low-latency and data integrity are of utmost importance.

Keywords - Real-Time Reconciliation, Anomaly Detection, Isolation Forest, Apache Kafka, Distributed Systems, Data Drift.

I. Introduction

The modern financial systems paradigm has radically realigned the architectural paradigm by changing monolithic architecture to distributed microservices to attain scalability, fault resistance, and operational responsiveness [1], [4]. Although this merger allows the independent scaling of the components, it brings about deep issues related to data management, especially in this context of consistency across heterogeneous data stores [2], [6]. The general lifecycle of a financial transaction would require a Payment Gateway to store a transaction in their internal database and at the same time provide a update to a secondary Internal Ledger service. This Dual-Write situation forms a period of vulnerability in which network partitions, crash of services or lapses in latency may lead to data drift a state where the payment has been charged as successful in one system but has not been charged or has not been charged at all in the other [3], [7]. Conventionally, financial institutions have used the "Strong Consistency" models (ACID transactions) to avoid such discrepancies. Nonetheless the consistency in high-load distributed systems can be very high at the expense of unacceptable latency penalties, which breaks the availability guarantees of high-frequency trading systems and real-time payment systems of the present day [5], [12]. As a result, a significant number of systems have switched to the use of the Eventual Consistency models, which are based on asynchronous data reconciliation processes [8], [9]. These reconciling processes are normally batch-based processes, which run at the end of a working day. Such latency causes financial institutions to become vulnerable to intraday fraud, error of operation and deviation that compounds with time [13], [30].

To overcome the latency of the batch processing, the industry is moving towards using Event-Driven Architectures (EDA) based on streaming systems with high throughput, such as Apache Kafka [10], [18]. Although EDA is a much faster way to synchronize data, it creates another problem that requires the maintenance of data integrity in real-time. Existing real-time validation systems are largely based on a form of static rule-based logic (e.g., Flag if transaction > $10,000) [20], [21]. Although useful in the detection of catastrophic failures or Hard Drifts (e.g. a 50% error), the static rules are fragile in nature. They do not have the flexibility to not only see low-level data corruptions (so-called Soft Drifts) such as a 0.5 percent to 3 percent data distortion which resembles normal noise but leads to huge financial losses when compounded but are hard to detect--they do not prevent them [24], [29]. The current paper suggests a new financial reconciliation system that is in real-time and combines the low-latency features of Apache Kafka with the reliability of Unsupervised Machine Learning. In contrast to rule-based systems, where one is required to have predefined thresholds, we apply an Isolation Forest algorithm to learn the regular behavior of the streams of transactions and identify anomalies by magnitude; instead, we look at the variance to detect anomalies. We make the following contributions:

- **Real-Time Architecture:** We show a stream-processing pipeline that can reconcile transactions at steady-state latency of about 3.76 ms and hence can be used in a high-frequency environment [15].
- **Soft Drift Detection:** We give a practical demonstration that, even though the traditional rules are unable to detect such inputs (0% recall on 3% drift), the proposed unsupervised model can report 100% recall on the same inputs.
- **Comparative Analysis:** We provide a quantitative comparison between rule-based baselines and the suggested solution based on AI, where we identify a trade-off between precision and recall in financial cases.

SoutheastCon 2026 | 979-8-3315-4642-7/26/$31.00 ©2026 IEEE | DOI: 10.1109/SOUTHEASTCON63549.2026.11476437

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:45:29 UTC from IEEE Xplore. Restrictions apply.

II. Literature Survey

Real-time reconciliation is a problem that lies at the nexus of distributed systems engineering, stream processing and anomaly detecting. This section is a literature review on these three areas with which the proposed framework can be contextualized.

A. Consistency of Data in Distributed Microservices.

The shift of old enterprise code to microservices has required the reconsideration of the data consistency pattern. Nookala [1] points out that it is true that microservices provide better scalability; however, they disaggregate the data flow, thus ACID guarantees are hard to preserve without paying a substantial locking cost. Furda et al. [3] and Laigner et al. [2] also explain that the issue of state across multitenant, stateless services tends to have the Dual-Write problem, where updates to two or more systems are performed concurrently die atomicity. To address this, Braun et al. [5] and Malygin [7] talk about design patterns employed in managing consistency in data-intensive systems. According to them, the only way to create bottlenecks is to strive to force high-load environments to high consistency. Rather, methods Pearl [9] and Gomes et al. [12] promote are asynchronous programming and Strong Eventual Consistency where systems can temporarily diverge but converge. But, as Borio [13] points out when dealing with financial systems, even temporary divergence can cause distress unless monitored, so there must be a mechanism which is not only eventually consistent, but also observably consistent in real time [19].

B. Event-Driven Architecture and Latency.

Researchers have resorted to the use of Event-Driven Architectures (EDA) in order to attain observability and accelerated convergence. Odofin et al. [18] and Cruz et al. [11] show that EDA, especially when deployed with message brokers such as Apache Kafka and RabbitMQ, is highly helpful in enhancing the operational agility of Fintech operations. Latency is the key measure of such architecture. Anon [14] gives a detailed discussion on real-time processing architectures and it has been found that applications that have high throughput demand that the processing time taken should be less than a second. Specifically, Karpiuk et al. [15] discuss Apache Kafka, confirming that it is suitable in low-latency activities like image processing, which can be compared to the high-volume needs of financial transaction monitoring. Moreover, Beereddy [10] presents Self-Healing patterns in streaming architectures, which states that the streams can not only carry data but also serve as a control plane to rectify errors. This is what our work is based on as the stream is used to identify errors (reconciliation) but not merely to carry data.

C. Static Rule-Based Systems Limitations.

Rule-based expert systems have been the standard in the fraud detection and reconciliation industry because of their ability to be interpreted [20]. Islam et al. [21] define models based on rules where rules are codified by domain people (e.g., IF amount_diff > 5% THEN alert). Nevertheless, this approach is reaching its limitations in literature. According to Roldan-Garcia et al. [20], rule-based systems are weak in terms of semantic consistency and are not flexible. However, more importantly, Clark et al. [24] and Ali et al. [25] speak about the challenge of Adaptive Thresholding. A fixed threshold can be either overshooting (in dynamic financial markets) or undershooting (resulting in alert fatigue). Hybrid methods that may combine rules with ML to enhance interpretability are suggested by Mohite and Ouarbya [22], [23], but again tend to use supervised learning that needs labeled data, which is not often available in new types of anomalies.

D. Unsupervised Anomaly detection and concept drift..

To overcome the inflexibility of rules, recent studies have been concerned with the unsupervised learning. Stefansson [27] discusses the question of unsupervised detection in financial transactions, where anomalies (fraud or error) are the outliers of a high-dimensional feature space. Isolation Forest algorithm is one of the most promising candidates for this task. Togbe et al. [26] and Zahoor et al. [28] show that Isolation Forests are more computationally efficient (linear complexity) and resource-efficient on streaming data, which is more effective than complex models such as One-Class SVMs in resource-constrained settings. This performance is essential in terms of sustaining the low latency, which is one of the conditions stated by Karpiuk et al. [15]. Lastly, there is the problem of the so-called Concept Drift, i.e. the phenomena of the statistical characteristics of data varying with time. Mhalla and Saied [31] and Lukats et al. [29] address the problem of continuous stream drift detection. Concept drift in our case, concept drift is Soft Drift (subtle corruption). These ideas are combined by Arzu et al. [16] and Liu et al. [17] into real-time fraud detectors. The work is furthered in our paper, which uses Isolation Forests to the reconciliation problem, aiming to locate the so-called Soft Drifts that our static rules, discovered in [20] and [21], fail to detect.

III. METHODOLOGY

This section elaborates the architectural design, implementation strategies, and experimental procedures that are used to test the proposed AI-driven reconciliation framework. The methodology is meant to model a high throughput distributed financial system where data integrity faces catastrophic failures (Hard Drifts) and subtle data corruption (Soft Drifts). The system has a stream-first implementation strategy, which focuses on ingestion of low-latency and real-time inference rather than traditional batch processing.

A. System Architecture

The suggested framework adheres to an Event-Driven Architecture (EDA) pattern that separates the source of transactions generation and the reconciliation logic. The system has three main different components that are the Data Simulator (Producer), the Message Broker (Kafka) and the Stream Processor (Faust/AI).

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:45:29 UTC from IEEE Xplore. Restrictions apply.

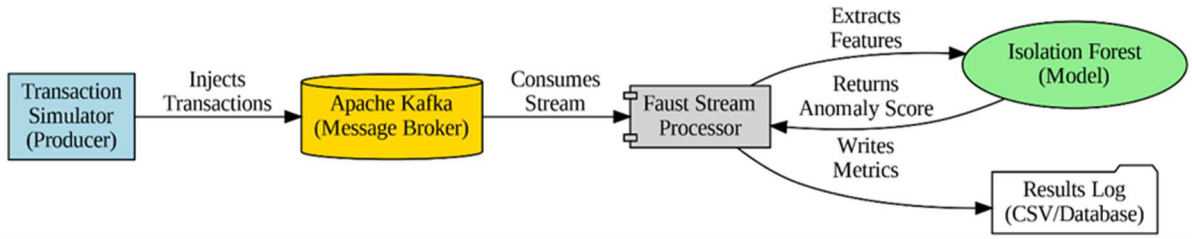


Fig. 1: Details the logic flow for the data generator

The top-level data flow would replicate a scenario of Dual-Write which is typical of microservices. One financial occurrence initiates two different messages, one of them being the Payment Gateway (Source of Truth) and the other one is the Internal Ledger. These messages are transmitted through the system asynchronously and they are balanced by a downstream consumer. The Transaction Simulator is the ingress point, which produces fake payloads of the JSON format with transaction IDs, timestamps and amounts. Such payloads are posted to the financial transaction topic of the Apache Kafka cluster. A Python library streaming library called Faust Stream Processor subscribes to this topic. In contrast to stateless consumers, Faust agent has a local state (windowing) to match Gateway and Ledger events and then feed the aggregated feature vector to the Isolation Forest model to detect anomalies.

B. Data Synthetic Generation and Drift Injection.

To test the robustness of the system very rigorously, we created a stochastic data generator which is able to create realistic financial traffic and individual error modes. The generator works in an endless cycle, creating paired events (GATEWAY and LEDGER) in accordance with a specific transaction ID. The Drift Injection Strategy is the main idea of our approach. Instead of depending on random noise, we designed two target scenarios of drift to measure the failure points of static rule-based systems:

- **Scenario A:** Hard Drift (Catastrophic Failure): a 20 percent change in the number of transactions. This is simulating large scale system failures, like currency conversion errors or partial writing failures. The scenario is crafted in such a way that both AI and fixed rules are easy to detect it.
- **Scenario B:** Soft Drift (Subtle Corruption): A one-third deviation (i.e., the change in 100 into 97). This emulates small bugs, including round-off errors or injurious salami slicing attacks. This situation is tailored to be below the conventional 5% index of most legacy rule-based systems.

The generator also adds a label Ground Truth to the Ledger message (drift injection). This label is also not visible in the reconciliation logic when processing but is applied in the post-hoc analysis to compute Precision and Recall.

C. Stream Processing and State Management

Faust-streaming is a stateful processing engine that is used to construct the reconciliation engine to process Kafka topics. This is a major problem with distributed reconciliation the Gateway event and the Ledger event of the same transaction ID are seldom at the same milliseconds.

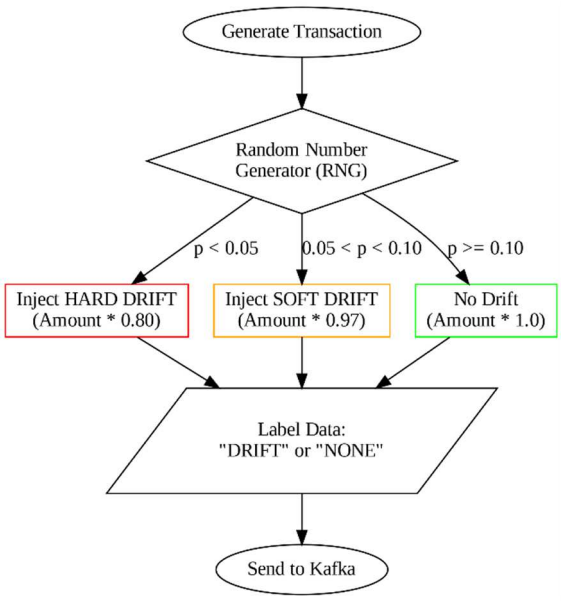


Fig. 2. Details the logic flow for the data generator

To handle this, Tumbling Window approach with an in-memory table (txt store) has been used. When an event arrives. The system will verify whether the Transaction ID is in the txt store.

- **In case No:** The event is saved in the memory awaiting its counterpart.
- **In case Yes:** The counterpart stored is recovered, making a complete matched pair. The system then computes the latency (difference between arrival time and creation time) and goes on to feature engineering.

To avoid the AI model collapsing because of zero variance (Gateway = Ledger), we add microscopic noise (uniform distribution between 0 and 10^{-5}) to the feature engineering process. This makes sure the Isolation Forest can construct sound decision trees even if the data is mathematically perfect.

D. Unsupervised Anomaly Detection Model

The Heart of the analytical engine is an Isolation Forest, one of the unsupervised learning algorithms that are selected due to computational efficiency (complexity: O(n)) and performance on high-dimensional streaming data. Isolation Forest operates through random partitioning of the feature space as opposed to distance-based approaches (e.g., K-Nearest Neighbors) that need computationally intensive distance computation on each point to obtain a result.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:45:29 UTC from IEEE Xplore. Restrictions apply.

1. Feature Engineering

The raw transaction data is converted to a feature vector which is acceptable to the model. We have taken two main features of each transaction pair:

- **Absolute Difference:**|Amount $_{Gateway}$ Subtract Amount $_{Ledger}$|
- **Ratio:** Amount $_{Ledger}$ / Amount $_{Gateway}$

2. Cold Start vs. Steady State Training

Another critical methodological issue was the setting up of the unsupervised model. Because the Isolation Forest needs a baseline distribution to detect outliers, the system has a Cold Start phase.

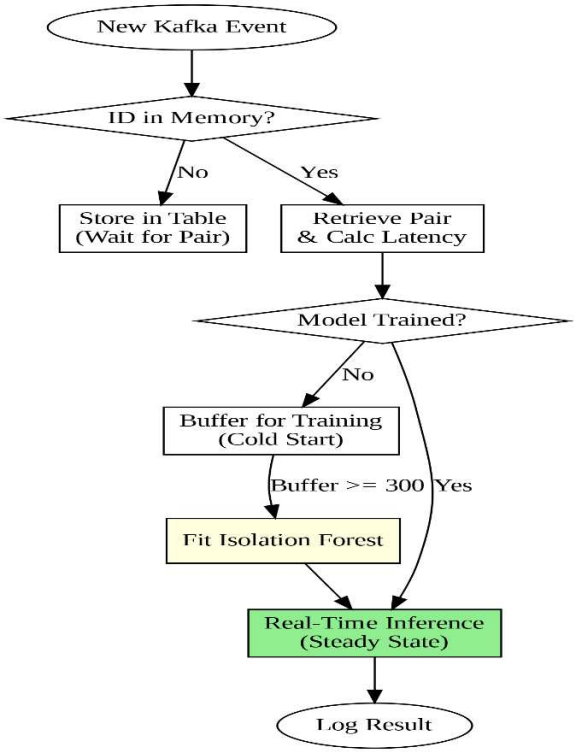


Fig. 3. Illustrates the internal logic of the Reconciler Agent

A buffering of the initial transactions, N=300, are placed in a training array. The inference is suspended at this stage. After filling the buffer, the model runs a fit operation on this first batch. After this initial setup, the system goes into a Steady State mode, in which thereafter all of the transactions are handled on a case one at a time basis in real-time (predict ()). This combination method enables the system to learn the peculiarities of the statistical distribution of the stream of transactions without pre-training with offline data.

E. Experimental Design and Assessment Measures

In order to compare the performance of the proposed AI-driven system with another one, we created a Baseline Rule-Based System that would operate in parallel. The standard industry logic is reflected in the next base: the gap between two transactions is considered as an anomaly only when it surpasses a set threshold of 5%. The experiment was run on an environment deployed locally on a Docker container to greatly reduce the network jitter whereby the latency was measured on a localized environment that depended on the processing overhead of the Python application but not the external network conditions. The simulation was able to come up with more than 6,000 transactions, with a mixture of,

- 90% Normal Transactions (Clean)
- 5% Hard Drifts (20% error)
- 5% Soft Drifts (3% error)

F. Evaluation Metrics

The metrics used to assess the performance of the system are:

- **End-to-End Latency:** This is the duration of time taken between the time the first event was generated, and the reconciliation logic was completed. This is a check of the timeliness of the system.
- **Recall (Sensitivity):** The probability of injected drifts (Hard and Soft) which the system correctly detected. This is the first measure of data integrity.
  Recall = True Positives / (True Positives + False Negatives)
- **Precision:** The proportion of the total flagged anomalies that were real drifts.
- **F1-Score:** This is a combination of Precision and Recall that shows an equal balance of the model accuracy.

The findings of this comparative study such as the detection rates of Soft Drifts and the latency distribution analysis are given in the subsequent chapter.

IV. RESULTS

The benchmarking of the two key performance aspects is the real-time processing rate (latency) and the accuracy of detection of anomalies in different drifting conditions. There were 6,166 transactions that were processed by the system, the initial 1,450 of which were used to initialize the model (Cold Start) and the rest 4,716 of which were part of the steady-state evaluation set.

A. Latency and Scalability in the system.

One of the major issues related to the implementation of ML into financial pipelines is the possibility of an additional latency. The temporal behavior of the system is shown in figure 4. The log scale shows a clear "Cold Start" period of time in which the latency shot up as the Isolation Forest buffered training data. Nevertheless, after several initializations (marked by vertical blue line) the system was instantly attracted to a stable processing state. The system attained mean end-to-end latency of 3.76 ms in the steady state and 95th percentile of 4.85 ms. As is illustrated in the distribution histogram in Figure 5, the processing time is tightly distributed as normal, which justifies the fact that the inference overhead of the Unsupervised Model is deterministic and negligible.

B. Comparative Classification Performance

The main experiment was that of a direct comparison between a standard Rule-Based logic (Threshold > 5%) and the suggested Isolation Forest model. The findings, as depicted in the Confusion Matrices in Figure 6 show a huge performance divergence.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:45:29 UTC from IEEE Xplore. Restrictions apply.

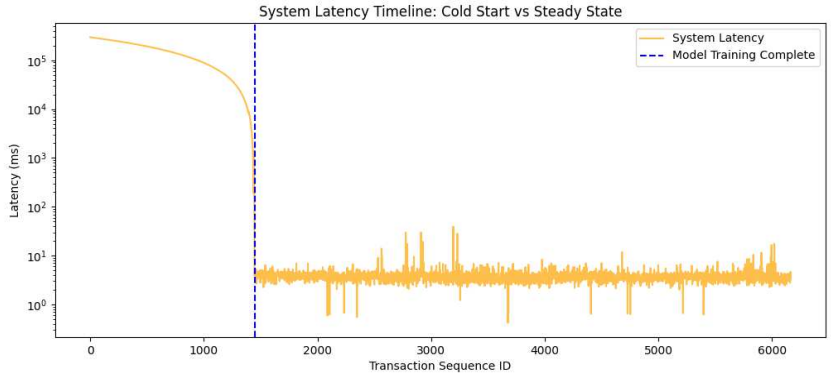


Fig. 4. System latency timeline. The transition between the training phase and the inference phase is indicated by the dashed line

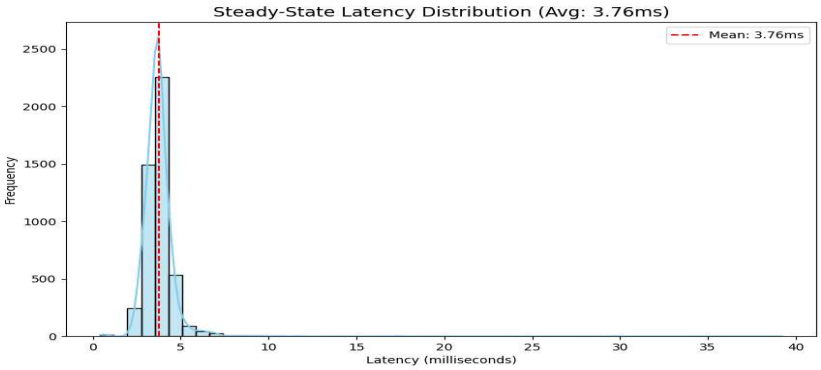


Fig. 5. Steady-state latency distribution. The system has sub-5ms latency on 95% of the transactions

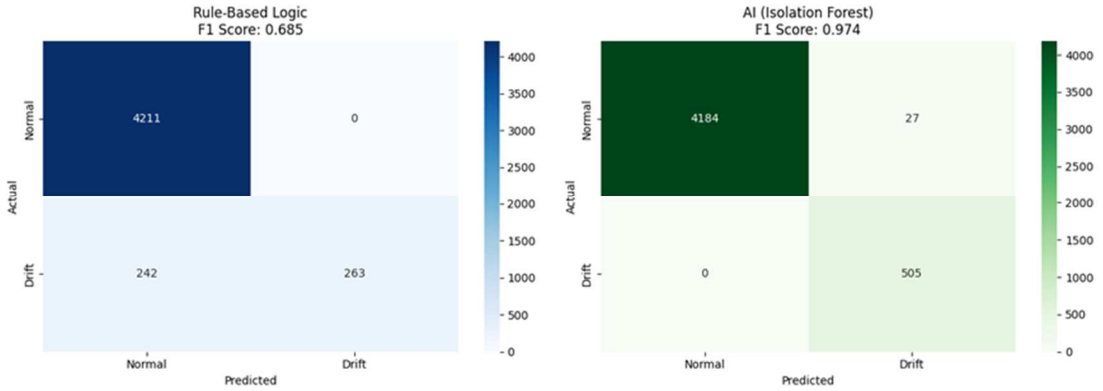


Fig. 6. Rule-Based Logic (Left) vs. AI Model (Right) Confusion Matrices

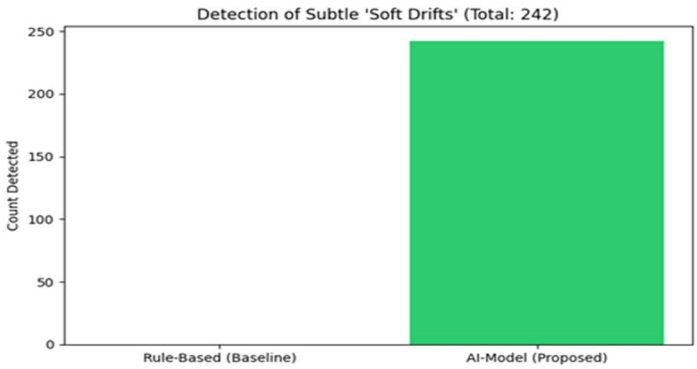


Figure 8 Subtle Soft Drifts Are Detected. The AI model is able to detect anomalies that evade fixed thresholds.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:45:29 UTC from IEEE Xplore. Restrictions apply.

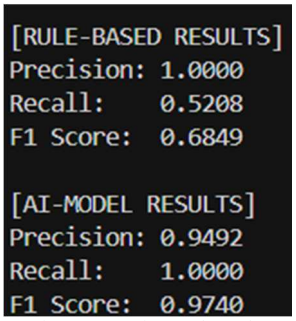


Fig. 7: Experimental results

The experimental results are showing in Fig. 7. The Precision of the Rule Based system was 1.00 with Recall of 0.52. This means that it produced no false alarm, but it was unable to identify almost half of the real anomalies. Conversely, the AI Model scored a Recall of 1.00 and was able to detect all the anomalous transactions in the dataset although with a minor decrease in Precision (0.95). As a result, the AI model had an F1-Score of 0.974 which is a 28.9 percent improvement compared to the baseline F1-Score of 0.685.

C. *Soft Drift Detection Capability*

The difference in the Recall values is described due to the detection rates of "Soft Drifts" (3% deviation). The Rule-Based system did not detect any single case of soft drift (0/242 detected) as shown in Figure 8. The difference was too slight to cause the movement of the statical threshold. On the contrary, the Isolation Forest detected 100 percent (242/242) of these subtle corruptions.

V. DISCUSSION

This confirms the hypothesis that unmanaged learning has the potential to increase the integrity of data in the distributed systems without affecting the constraints imposed by real-time performance.

A. *Trade-off between Precision and Recall*

The Rule-Based system exhibited ideal Precision (1.0) which is frequently mentioned as an advantage of deterministic logic. Nonetheless, the price of this accuracy was a disastrous flunky in Recall (0.52). Financially, a False Negative (lost fraud or error) is far much more harmful than a False Positive (a false alarm). The proposed AI model achieved a small trade-off to produce 27 False Positives (Precision 0.95) to have the perfect Recall. This is a better outcome operationally; it is operationally cost-efficient to check 27 innocent transactions, but it is an operationally expensive to miss 242 erroneous transactions.

B. *Concept Drift Efficacy.*

The vulnerability of the static thresholds to so-called Soft Drifts is highlighted by the fact that the Rule-Based system failed entirely to identify such issues. Salami slicing attacks and floating-point bugs that are difficult to detect are often based on the fact that they remain undetected by such levels. The Isolation Forest has been successful as it does not seek magnitude, it seeks variance. Although 3 percentage points is minimal, it breaks the statistical approach between the Gateway and Ledger amounts that will be created during the training stage. This demonstrates that unsupervised models are resilient to Concept Drift in which the characteristic of the error is different, but the statistical aberration is retained.

C. *High-Frequency Trading (HFT) Feasibility.*

The latency evaluation verifies the feasibility of Python-application based stream processing to near-real-time applications. This framework is already in the range of the requirements of payment reconciliation, fraud detection, and post-trade processing as it has a steady-state latency of 3.76 ms. Although C++ or Java implementations may result in lower latency of a sub-milliseconds, the ease of implementation and the flexibility of the Scikit-Learn ecosystem is a strong appeal when it comes to rapid development and deployment.

VI. CONCLUSION

The problem of the so-called Dual-Write consistency associated with distributed microservices involved in this paper was the proposal of a real-time reconciliation system that combines Apache Kafka and Unsupervised Machine Learning.

Superior Detection: The Isolation Forest model has been superior to traditional static rules, achieving 100% Recall on catastrophic failures and subtle data corruption, eliminating the blindness to subtle data corruption of the old systems of the Soft Drift.

Operational Viability: This system demonstrated a steady-state processing latency of 3.76 ms, confirming that the high-speed financial pipelines can be enabled with sophisticated anomaly detection without introducing bottlenecks in high-throughput financial pipelines.

Adaptive Resilience: The unsupervised model had estimated the statistical baseline of the transaction stream, which, contrary to the rule-based system, it was able to automatically adjust its threshold requiring a self-healing data consistency mechanism.

Future Work: Future directions will be to research in two directions: applying On-line Learning algorithms (e.g., Hoeffding Trees) so that the Cold Start phase can be completely removed and deploy the framework on Kubernetes to test horizontal scalability at terabyte scale. The present research provides a base reference point of deploying AI-based observability as part of the critical path of financial infrastructure.

REFERENCES

[1] Guruprasad Nookala, "Microservices and Data Architecture: Aligning Scalability with Data Flow," *International Journal of Digital Innovation*, vol. 4, no. 1, 2023, Accessed: Feb. 05, 2025. [Online]. Available: https://researchworkx.com/index.php/ijdi/article/view/19

[2] R. Laigner, Y. Zhou, M. Antonio, Y. Liu, and M. Kalinowski, "Data Management in Microservices: State of the Practice, Challenges, and Research Directions," *arXiv.org*, 2021. https://arxiv.org/abs/2103.00170 (accessed Dec. 06, 2025).

[3] A. Furda, C. Fidge, O. Zimmermann, W. Kelly, and A. Barros, "Migrating Enterprise Legacy Source Code to Microservices: On Multitenancy, Statefulness, and Data Consistency," *IEEE Software*, vol. 35, no. 3, pp. 63–72, May 2018, doi: https://doi.org/10.1109/ms.2017.440134612.

[4] G. Liu, B. Huang, Z. Liang, M. Qin, H. Zhou, and Z. Li, "Microservices: architecture, container, and challenges," *2020 IEEE 20th International Conference on Software Quality, Reliability and*

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:45:29 UTC from IEEE Xplore. Restrictions apply.

Security Companion (QRS-C), Dec. 2020, doi: https://doi.org/10.1109/qrs-c51114.2020.00107.

[5] S. Braun, S. Deßloch, E. Wolff, F. Elberzhager, and A. Jedlitschka, "Tackling Consistency-related Design Challenges of Distributed Data-Intensive Systems - An Action Research Study," *arXiv:2108.03758 [cs]*, Aug. 2021, doi: https://doi.org/10.1145/3475716.3475771.

[6] Y. Wang, H. Kadiyala, and J. Rubin, "Promises and challenges of microservices: an exploratory study," *Empirical Software Engineering*, vol. 26, no. 4, May 2021, doi: https://doi.org/10.1007/s10664-020-09910-y.

[7] Malygin Dmitriy, "Addressing data consistency in high-load distributed systems: implementation challenges and solutions," *Universum: технические науки*, vol. 7, no. 10 (127), 2024, Accessed: Jun. 08, 2025. [Online]. Available: https://cyberleninka.ru/article/n/addressing-data-consistency-in-high-load-distributed-systems-implementation-challenges-and-solutions

[8] O. V. Talaver and T. A. Vakaliuk, "Reliable distributed systems: review of modern approaches," *Journal of Edge Computing*, vol. 2, no. 1, pp. 84–101, May 2023, doi: https://doi.org/10.55056/jec.586.

[9] I. A. Pearl, "Evaluation of the applicability of asynchronous programming methods to the data consistency problem in a microservices environment," *Journal Scientific and Technical Of Information Technologies, Mechanics and Optics*, vol. 134, no. 4, pp. 473–481, Aug. 2021, Accessed: Dec. 06, 2025. [Online]. Available: https://ntv.ifmo.ru/en/article/20564/ocenka_primenimosti_metodov_asinhronnogo_programmirovaniya_pri_reshenii_problemy_soglasovannosti_dannyh_v_mikroservisnoy_srede.htm

[10] Karthik Reddy Beereddy, "Self-Healing Streaming Architecture: Resilience Patterns for Event-Driven Microservices," *Journal of Computer Science and Technology Studies*, vol. 7, no. 8, pp. 254–268, Jul. 2025, doi: https://doi.org/10.32996/jcsts.2025.7.8.30.

[11] C. D. Cruz, M. Santos, and R. Navarro, "Improving Operational Agility in Enterprises Through Event Driven Approaches to Data Integration," *Journal of Applied Computational Science, Numerical Methods, and Scientific Computing in Engineering*, vol. 15, no. 10, pp. 1–13, 2025, Accessed: Dec. 06, 2025. [Online]. Available: https://soloncouncil.com/index.php/JACSNMSCE/article/view/2025-OCT-04

[12] V. Gomes, M. Kleppmann, D. P. Mulligan, and A. R. Beresford, "Verifying strong eventual consistency in distributed systems," *Proceedings of the ACM on programming languages*, vol. 1, no. OOPSLA, pp. 1–28, Jul. 2017, doi: https://doi.org/10.1145/3133933.

[13] C. E. V. Borio, "Change and Constancy in the Financial System: Implications for Financial Distress and Policy," *SSRN Electronic Journal*, 2007, doi: https://doi.org/10.2139/ssrn.1022874.

[14] S. anon, "A Comprehensive Analysis of Real-time Data Processing Architectures for High-throughput Applications," *SSRN Electronic Journal*, Jan. 2025, doi: https://doi.org/10.2139/ssrn.5034117.

[15] N. Karpiuk, H. Klym, and T. Tkachuk, "USAGE OF APACHE KAFKA FOR LOW-LATENCY IMAGE PROCESSING," *Electronics and Information Technologies*, vol. 26, Jan. 2024, doi: https://doi.org/10.30970/eli.26.5.

[16] F. Arzu *et al.*, "Real-Time Financial Fraud Detection: An Intelligent Data-Driven Framework Integrating Machine Learning, Stream Processing, and Big Data Analytics for High-Velocity Transaction Monitoring," *The Asian Bulletin of Big Data Management*, vol. 5, no. 4, pp. 124–154, Nov. 2025, doi: https://doi.org/10.62019/f3c0g313.

[17] C. Liu, H. Tang, Z. Yang, K. Zhou, and S. Cha, "Big Data-Driven Fraud Detection Using Machine Learning and Real-Time Stream Processing," *arXiv.org*, 2025. https://arxiv.org/abs/2506.02008

[18] O. T. Odofin, S. Owoade, E. Ogbuefi, J. C. Ogeawuchi, O. S. Adanigbo, and T. P. Gbenle, "Integrating Event-Driven Architecture in Fintech Operations Using Apache Kafka and RabbitMQ Systems," *International Journal of Multidisciplinary Research and Growth Evaluation*, vol. 3, no. 4, pp. 635–643, 2022, doi: https://doi.org/10.54660/.ijmrge.2022.3.4.635-643.

[19] A. Chavan, "Managing Scalability and Cost in Microservices ArchitectureBalancing Infinite Scalability with Financial Constraints," *Journal of Artificial Intelligence & Cloud Computing*, pp. 1–14, Dec. 2023, doi: https://doi.org/10.47363/jaicc/2023(2)e264.

[20] M. del M. Roldán-García, J. García-Nieto, and J. F. Aldana-Montes, "Enhancing semantic consistency in anti-fraud rule-based expert systems," *Expert Systems with Applications*, vol. 90, pp. 332–343, Dec. 2017, doi: https://doi.org/10.1016/j.eswa.2017.08.036.

[21] S. Islam, Md. M. Haque, and A. N. M. Rezaul Karim, "A rule-based machine learning model for financial fraud detection," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 14, no. 1, p. 759, Feb. 2024, doi: https://doi.org/10.11591/ijece.v14i1.pp759-771.

[22] R. Mohite and L. Ouarbya, "Interpretable Anomaly Detection: A Hybrid Approach Using Rule-Based and Machine Learning Techniques," *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)*, pp. 1–10, Apr. 2024, doi: https://doi.org/10.1109/i2ct61223.2024.10543396.

[23] L. Ouarbya and M. Rahul, "Interpretable Anomaly Detection: A Hybrid Approach Using Rule-Based and Machine Learning Techniques - Goldsmiths Research Online," *Gold.ac.uk*, Dec. 2023, doi: https://research.gold.ac.uk/id/eprint/34694/1/Interpretable_Anomaly_Detection__A_Hybrid_Approach_Using_Rule_Based_and_Machine_Learning_Techniques.pdf.

[24] J. Clark, Z. Liu, and N. Japkowicz, "Adaptive Threshold for Outlier Detection on Data Streams," *2022 IEEE 9th International Conference on Data Science and Advanced Analytics (DSAA)*, pp. 41–49, Oct. 2018, doi: https://doi.org/10.1109/dsaa.2018.00014.

[25] M. Ali, Ehab Al-Shaer, H. Khan, and Syed Ali Khayam, "Automated Anomaly Detector Adaptation using Adaptive Threshold Tuning," *ACM Transactions on Information and System Security*, vol. 15, no. 4, pp. 1–30, Apr. 2013, doi: https://doi.org/10.1145/2445566.2445569.

[26] Maurras Ulbricht Togbe *et al.*, "Anomaly Detection for Data Streams Based on Isolation Forest Using Scikit-Multiflow," *Lecture notes in computer science*, pp. 15–30, Jan. 2020, doi: https://doi.org/10.1007/978-3-030-58811-3_2.

[27] Hafsteinn Atli Stefánsson 1996, "Unsupervised anomaly detection in financial transactions," *Skemman.is*, 2023. https://skemman.is/handle/1946/44727 (accessed Sep. 29, 2025).

[28] A. Zahoor, W. Abbasi, M. Z. Babar, and A. Aljohani, "Robust IoT security using isolation forest and one class SVM algorithms," *Scientific Reports*, vol. 15, no. 1, Oct. 2025, doi: https://doi.org/10.1038/s41598-025-20445-4.

[29] D. Lukats, O. Zielinski, A. Hahn, and F. Stahl, "A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams," *International Journal of Data Science and Analytics*, Aug. 2024, doi: https://doi.org/10.1007/s41060-024-00620-y.

[30] J. Arvola, "Enhancing error process handling in account reconciliation," *Theseus.fi*, 2024, doi: https://www.theseus.fi/handle/10024/872620.

[31] H. Mhalla and M. A. Saied, "Detecting Continuous Integration Skip : A Reinforcement Learning-based Approach," *arXiv.org*, 2024. https://arxiv.org/abs/2405.09657 (accessed Dec. 06, 2025).

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:45:29 UTC from IEEE Xplore. Restrictions apply.